

Mini-Project-I — Project Summary

Course: PCAM ZC412 — Machine Learning Engineering **Flavor:** A — NYC Taxi ETA Prediction

Team: Kishore Nandhalu | Vinay | Vishruth **Repository:** <https://github.com/2025paml539-lab/mleng-mini-project>

PROBLEM

Predict NYC taxi trip duration (in seconds) from pickup/dropoff GPS coordinates, time of day, day of week, and computed distance. The dataset is the NYC Taxi Trip Duration competition dataset from Kaggle — 1,458,644 trips recorded between January and June 2016.

WHAT WE BUILT

A four-stage ML pipeline following the course module sequence $M2 \rightarrow M3 \rightarrow M4 \rightarrow M5$: data ingestion and validation, model training with experiment tracking, REST API serving, and prediction monitoring with drift detection.

Week 1 — Data Engineering (M2)

What we did

- Loaded raw CSV (1,458,644 rows, 11 columns) using `pipeline/ingest.py`
- Ran 4-level validation in `pipeline/validate.py` — removed 689 outlier rows (0.047%)
- Engineered 5 features in `pipeline/features.py` and saved to `data/processed/features.csv`
- Versioned the dataset using DVC — only the pointer file (`train.csv.dvc`) is in Git, not the 191MB CSV

Validation results

Level	Check	Result
L1 Schema	All 11 columns, correct types, 0 nulls	PASS

L2 Range	Passenger count 1–6, GPS within NYC, duration 1–86400s	WARN — 689 rows removed
L3 Statistical	Row count > 1M, mean duration 300–1500s	PASS
L4 Business	dropoff timestamp > pickup timestamp	PASS

Features engineered

Feature	How	Why
hour_of_day	Extract hour from pickup_datetime	Traffic patterns differ by hour
day_of_week	Extract weekday (0=Mon, 6=Sun)	Weekday vs weekend travel patterns differ
is_weekend	1 if Saturday or Sunday	Leisure trips vs commute trips behave differently
distance_km	Haversine formula on GPS coords	Strongest direct predictor of trip duration
pickup_hour_bin	night / morning / afternoon / evening	Captures non-linear time-of-day effects

All transformation parameters (mean, std, bin boundaries) are saved to `artifacts/feature_schema.json` and reused at serving time to prevent training-serving skew.

Week 2 — Experimentation & Reproducibility (M3)

What we did

- Trained two models using `training/train.py` with all hyperparameters in `params.yaml`
- Used a temporal 80/20 split — no shuffling, to respect time order of trips
- Log-transformed the target (`log1p(trip_duration)`) to reduce skew
- Logged both runs to MLflow experiment `eta-prediction` with params, metrics, and model artifacts

- Selected best model based on R^2 improvement threshold of 0.05

Model comparison

Metric	Linear Regression	XGBoost
RMSE (seconds)	282,405,528	3,162
MAE (seconds)	525,057	344
R^2	0.3883	0.6532
Training time	~2 sec	~12 sec

Why XGBoost was selected

Linear Regression failed badly because trip duration is not linearly related to distance. A 2km trip during rush hour can take 40 minutes while the same route at 2am takes 8 minutes. Linear models cannot capture this interaction. XGBoost handles it naturally through tree splits. R^2 improvement was 0.2649, well above the 0.05 threshold.

Reproducibility

Re-running `python training/train.py` with `random_state=42` and the same `data/processed/features.csv` produces identical metrics every run. Verified manually.

MLflow runs

Run name	RMSE	MAE	R^2	Status
linear_regression	282,405,528 s	525,057 s	0.3883	FINISHED
xgboost	3,162 s	344 s	0.6532	FINISHED

XGBoost hyperparameters: `n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, `random_state=42`

Week 3 — Model Packaging & Deployment (M4)

What we did

- Built a FastAPI serving endpoint in `serving/api.py`
- Model, scaler, and label encoder are loaded once at startup — not per request
- Input validation via Pydantic in `serving/schemas.py` — invalid inputs rejected before the model is called
- Wrote Dockerfile using `python:3.11-slim` base, non-root user, port 8000

API test results

Test	Input	HTTP	Result
GET /health	—	200	model_loaded=true, version=v1.0-week3
POST /predict	Night trip (00:30)	200	427.84s (7.1 min)
POST /predict	Morning rush (08:30)	200	1101.86s (18.4 min)
POST /predict	Weekend afternoon	200	1320.36s (22.0 min)
POST /predict	Evening long trip	200	1803.75s (30.1 min)
POST /predict	passenger_count=10	422	Validation error — rejected
POST /predict	longitude=-50.0	422	Validation error — rejected

Sample request

```
POST http://localhost:8000/predict
Content-Type: application/json

{
  "pickup_datetime": "2016-06-30 23:59:58",
  "pickup_longitude": -73.982155,
  "pickup_latitude": 40.767937,
  "dropoff_longitude": -73.964630,
  "dropoff_latitude": 40.765602,
  "passenger_count": 1
}
```

Response:

```
{
  "predicted_duration_seconds": 427.84,
  "predicted_duration_minutes": 7.13,
  "model_version": "v1.0-week3",
  "status": "ok"
}
```

The predictions make sense — same route takes 7 min at night vs 18 min during morning rush. The model picks up the hour_of_day signal correctly.

Week 4 — Monitoring, Drift & Retraining (M5)

What we did

- `monitoring/logger.py` — appends every prediction to `monitoring/prediction_log.csv` with timestamp, inputs, output, model version
- `monitoring/simulate_drift.py` — injected two realistic drift scenarios into the log
- `monitoring/drift_detector.py` — runs KS test (numerical) and Chi-squared test (categorical) to detect distribution shift

Drift simulation scenarios

Scenario	What changed	Records
Rush-hour surge	distance_km mean shifted 3.4km → 12.5km. Trips concentrated at hours 7–9am and 5–7pm only.	150
Festival / holiday	Very short distances (mean 1.2km). Late night only (10pm–2am). Groups of 3–6 passengers.	150
Normal baseline	Matching training distribution	300

Drift detection results (after simulation)

Feature	Test	Statistic	p-value	Result
distance_km	KS	0.3864	0.000000	DRIFT DETECTED
hour_of_day	KS	0.1775	0.000000	DRIFT DETECTED
passenger_count	Chi ²	112.36	0.000000	DRIFT DETECTED

Retraining trigger design

Trigger condition: distance_km KS p-value < 0.05 for 3 consecutive monitoring windows (each window = 500 new predictions).

We chose distance_km as the primary trigger because it has the highest feature importance in the XGBoost model (~0.55). Drift in this feature has the most direct impact on prediction quality.

Three consecutive windows prevents false triggers from one-off spikes.

Action when triggered: alert ML engineer → human reviews drift report → if confirmed, re-run `python training/train.py` with updated data → evaluate new model → promote if R² improves by > 0.02.

Repository Structure

```

mleng-mini-project/
├─ pipeline/
│   ├── ingest.py          – load and profile raw CSV
│   ├── validate.py        – 4-level data validation
│   └─ features.py         – feature engineering
├─ training/
│   └─ train.py            – LR + XGBoost, MLflow logging
├─ serving/
│   ├── api.py             – FastAPI /predict and /health
│   ├── schemas.py         – Pydantic input validation
│   └─ run_tests.py        – 7 API test cases
├─ monitoring/
│   └─ logger.py           – log every prediction to CSV

```

```
|   ├── simulate_drift.py  - inject drift scenarios
|   └── drift_detector.py  - KS test + Chi-squared test
└── reports/
    ├── model_comparison.md
    ├── api_test_report.md
    └── drift_report.md
└── outputs/
    ├── week1/  - 4 charts + pipeline run log
    ├── week2/  - 3 charts + 7 MLflow screenshots
    ├── week3/  - 3 charts + 3 API screenshots
    └── week4/  - 3 charts + 3 terminal screenshots
└── artifacts/
    ├── model.pkl, scaler.pkl, label_encoder.pkl
    ├── feature_schema.json, model_selection.json
└── data/raw/train.csv.dvc  - DVC pointer (dataset not in Git)
└── params.yaml              - all hyperparameters
└── dvc.yaml                 - pipeline stage definitions
└── requirements.txt
└── Dockerfile
└── README.md
```

Commit History (Weekly Progress)

Tag	Commit	Week
v1.0-week1	Data ingestion, validation, feature engineering, DVC versioning	Week 1
v1.0-week2	MLflow experiment tracking, LR vs XGBoost, model selected	Week 2
v1.0-week3	FastAPI endpoint, Dockerfile, 7 API tests passed	Week 3
v1.0-final	Monitoring logger, drift simulation, KS-test detection	Week 4

Unit Test Summary

Test file	Tests	Result
test_week2.py	22	22 passed
test_week3.py	22	22 passed
test_week4.py	24	24 passed
final_check.py	112	112 passed

Submission Details

#	Deliverable	Location
1	Versioned dataset + pipeline code	GitHub repo — pipeline/ , data/raw/train.csv.dvc
2	MLflow experiment logs + model comparison report	outputs/week2/ , reports/model_comparison.md
3	Working API endpoint + test calls	serving/ , reports/api_test_report.md
4	Monitoring log + drift report + retraining design	monitoring/ , reports/drift_report.md
5	README + architecture + demo video	README.md , outputs/HOW_TO_RUN.md , video